Reinforcement Learning for Swiss Post Package Sorting

Step-by-step deep explanation of your project: from problem framing to DQN, PPO, MaskablePPO, evaluation, visual simulation, and business recommendations. Based on RL_Project_Report_v1.6.pdf.

Prepared for: Moataz Mansour | Companion learning document

0. Big Picture - What Your Project Is

Your project turns a package sorting center into a Reinforcement Learning problem. The agent observes the operational situation, chooses an action, receives a reward, and gradually learns which actions reduce overload, queues, delay, and operational cost.

In simple words: you are not only predicting that congestion may happen; you are training an agent to decide what to do when congestion is building up.

Classic ML / Forecasting	Your RL Project
Predicts a value, e.g. delay in the next hour.	Chooses an intervention, e.g. add chute, reroute, priority, or wait.
Descriptive: tells what may happen.	Prescriptive: recommends an action.
Model learns from historical input-output pairs.	Agent learns by reward from simulated operational consequences.

1. Business Problem

Swiss Post sorting centers process many packages per hour across multiple chutes. During peak periods, some chutes become overloaded. Queue length increases, processing time rises, and delays can cascade. The business question is: can an RL agent learn when to intervene and when not to intervene?

The project compares rule-based logic, from-scratch RL agents, Stable-Baselines3 agents, and baselines. The target is lower delay and overload, while avoiding unnecessary operational cost.

2. RL Mapping: Sorting Center as an Environment

The most important step is the mapping from business process to RL concepts.

RL term	Meaning in your project
Environment	The simulated package sorting center. It receives demand, updates queues and load, and returns rewards.
Agent	The decision maker: Q-learning, SARSA, DQN, PPO, A2C, MaskablePPO, etc.
State	The current operational situation observed by the agent.
Action	One of the allowed interventions.
Reward	The score that tells the agent whether the action was good or bad.
Policy	The learned strategy: state -> best action.

3. State - What the Agent Sees

Your state is a compact numeric representation of the sorting center at a time step. In the report, the core state has six dimensions: chute load, hour of day, day of week, active chutes, queue length, and processing rate.

Feature	Example	Why it matters
Chute load	0.85	Shows how close the chutes are to overload.
Hour of day	0.52	Captures daily peak/off-peak patterns.
Day of week	0.4	Captures weekly demand differences.
Active chutes	0.5	Shows available capacity.
Queue length	0.6	Shows backlog level.
Processing rate	0.8	Shows throughput efficiency.

This state is what the agent uses to decide. It does not need the full database; it needs the features that summarize the current operational risk.

4. Actions - What the Agent Can Do

Your action space is discrete. This is important because it explains why DQN, PPO, A2C, and MaskablePPO make sense, while algorithms mainly designed for continuous actions such as TD3/DDPG are not the natural first choice.

Action ID	Action	Operational meaning
0	Do Nothing	No intervention. Good when the system is healthy.
1	Add Chute	Increase capacity by activating another chute.
2	Reroute	Reduce queue by diverting packages to another route/chute.
3	Priority Processing	Temporarily increase processing speed.
4	Remove Chute	Scale down capacity to save resources when load is low.

5. Reward Function - The Steering Wheel

The reward function is the most important design choice. It defines what good behavior means. In your project, reward combines load penalty, throughput bonus, overload penalty, action cost, and chute maintenance cost. You also added a bonus for smart inaction and a penalty for keeping excess chutes active when load is low.

```
reward = -load_penalty + throughput_bonus - overload_penalty
         - action_cost - chute_maintenance

bonus: +4.0 if load < 0.9 and throughput > 0.6
penalty: -2.0 per extra chute when load < 0.3
```

This means the agent is not rewarded for always doing something. It is rewarded for taking the right action at the right time. Sometimes the best action is Do Nothing or Remove Chute.

6. MDP - The Formal Foundation

The environment is modeled as a Markov Decision Process. This means the next state depends mainly on the current state and current action. The agent does not need to remember the entire past if the state contains enough useful information.

MDP component	Your implementation
State space S	[load, hour, day, chutes, queue, rate]
Action space A	{0:nothing, 1:add, 2:reroute, 3:priority, 4:remove}
Transition P(s_next | s,a)	Simulation physics: actions affect queue/capacity.
Reward R(s,a)	Delay, throughput, overload, action cost, chute penalty.
Discount gamma	0.99, meaning future rewards matter strongly.

7. Data Overview

The project uses data4day_with_issues.csv: four days of hourly Swiss Post package sorting data. The report states about 144,500 records before cleaning. After removing rows with negative processing times, the data is aggregated to chute-hour simulation states.

Column	Meaning
HOUR_TIME	Timestamp at hourly granularity.
CHUTE	Sorting lane identifier, e.g. R0101.
PACKAGE_COUNT	Packages in that hour/chute/ZIP combination.
AVG_PROCESSING_TIME_MINUTES	Average processing time.

The key idea: real historical demand drives the simulation. RL then learns interventions on top of this data-driven demand pattern.

8. Environment Versions: V1, V2, V3

Your project evolved through three environment designs. This is normal in RL: the first environment often teaches you what is missing.

Version	What changed	Lesson
V1	Raw demand, starting with 2 chutes, max 5, episode length 100.	Demand was too low relative to capacity. Agent could brute-force by adding chutes.
V2	Demand amplified 20x, starts with 1 chute, maintenance costs added.	Harder and more realistic, but some agents still learned poor single-action behavior.
V3	Action masking, smart inaction reward, curriculum learning.	Better learning because invalid actions are blocked and reward shaping is improved.

9. Action Masking and MaskablePPO

Action masking means the environment tells the agent which actions are valid at the current state. For example, if only one chute is active, Remove Chute may be invalid. If maximum chutes are already active, Add Chute may be invalid.

MaskablePPO improves learning because the agent does not waste time trying impossible actions. This is very useful for operational systems with hard constraints.

10. Curriculum Learning

Curriculum learning means you train the agent gradually: first easy scenarios, then medium, then hard. This is like teaching a person: first simple cases, then peak demand.

Phase	Difficulty	Demand scale	Steps	Purpose
1	Easy	5x	100K	Learn basic mechanics.
2	Medium	12x	100K	Learn timing.
3	Hard	20x	100K	Master peak demand.

11. Q-Learning From Scratch

Q-learning uses a Q-table. Each entry estimates the long-term value of taking an action in a state. Your discretized Q-table has 48,000 entries. The update uses the Bellman idea: current Q-value moves toward immediate reward plus best future value.

```
Q(s,a) = Q(s,a) + alpha * [reward + gamma * max_a' Q(s_next,a') - Q(s,a)]
```

Q-learning is off-policy: it learns the value of the best possible next action, even if the behavior during exploration was random.

12. SARSA From Scratch

SARSA is similar to Q-learning, but more conservative. It updates using the actual next action chosen by the current policy, not the theoretical best next action.

```
Q(s,a) = Q(s,a) + alpha * [reward + gamma * Q(s_next, actual_next_action) - Q(s,a)]
```

Aspect	Q-Learning	SARSA
Policy type	Off-policy	On-policy
Next value	Best next action	Actual next action
Behavior	More optimistic	More conservative
Operational interpretation	May learn aggressive interventions	May avoid risky transitions

13. DQN From Scratch - The Deep Learning Part

DQN replaces the Q-table with a neural network. This is the key deep learning part. Instead of storing Q-values for every possible state, the neural network learns to approximate Q-values.

In your report, the from-scratch DQN architecture is 7 -> 128 -> 128 -> 5 with ReLU activations. This means: input features go into two hidden layers of 128 neurons, and the output layer has 5 values, one Q-value per action.

```
Input state  ->  Neural Network  ->  [Q_do_nothing, Q_add, Q_reroute, Q_priority, Q_remove]

Agent chooses action = argmax(Q-values)
```

DQN is still reinforcement learning, but the neural network is trained using targets that look like supervised learning labels. The target comes from the Bellman equation.

14. DQN Components: Replay Buffer and Target Network

DQN needs extra tricks to train stably.

Component	Why it is needed
Experience Replay Buffer	Stores past transitions (state, action, reward, next_state). Training samples random mini-batches to avoid learning only from the latest sequence.
Target Network	A slower copy of the Q-network used to compute targets. This stabilizes learning.
Epsilon Decay	Starts with exploration, then gradually exploits learned policy.
Batch Training	Updates neural network using many transitions at once.

15. SB3 Training Strategy

Stable-Baselines3 gives tested implementations of RL algorithms. Your project trains DQN, A2C, PPO, and MaskablePPO for 300K steps. The temporal split trains on Days 1-2 and tests on Day 4, which checks if the learned policy generalizes to unseen future demand.

Algorithm	Type	Key idea
DQN	Value-based	Learns Q-values using replay buffer and target network.
A2C	Actor-Critic	Actor chooses actions; critic estimates value.
PPO	Policy Gradient	Directly optimizes policy with clipped updates for stability.
MaskablePPO	Constrained Policy Gradient	Same PPO idea, but invalid actions are blocked.

16. Hyperparameters

Your main hyperparameters are alpha/learning rate, gamma, and exploration/entropy settings.

Parameter	Meaning	In your project
alpha / learning rate	How strongly the model updates from new experience.	0.1 for tabular; 5e-4 for DQN; 3e-4 for PPO/MaskablePPO.
gamma	How much future reward matters.	0.99, so the agent cares about long-term consequences.
epsilon	Exploration rate for Q-learning/DQN.	Decays from 1.0 to 0.05.
entropy coef	Encourages policy exploration in PPO/A2C.	0.01.

17. Baselines and Why They Matter

Baselines tell you whether RL is actually useful. Your project compares learned agents with random, do-nothing, always-add-chute, and heuristic policies.

If RL cannot beat simple baselines, it is not ready. In your results, the from-scratch DQN slightly beats the heuristic on reward while keeping overload very low.

18. Results Interpretation

On the test set, the report shows DQN from scratch as the best reward: 1,864, compared with heuristic 1,834. Overload is 0.1% for DQN scratch and 0.6% for heuristic. This suggests the learned DQN policy is competitive with, and slightly better than, the hand-crafted policy.

Policy	Reward	Delay	Overload %
DQN scratch	1,864	0.49h	0.1%
Heuristic	1,834	0.47h	0.6%
Ensemble	1,593	0.65h	2.2%
DQN-v3 SB3	1,526	0.72h	3.9%
PPO-v3	1,500	0.44h	2.2%
MaskablePPO	1,481	0.62h	4.6%
A2C-v3	1,442	0.36h	1.1%

Important: reward is the main optimization metric, but delay and overload must also be checked. A high reward with unacceptable overload would be operationally risky.

19. Policy Heatmap

The policy heatmap is an interpretability tool. It asks: for different combinations of chute load and queue length, which action does the trained agent prefer? This helps you see the decision boundaries instead of treating the model as a black box.

Region	Expected action	Reason
Low load, low queue	Nothing or Remove Chute	System healthy; save resources.
High load, low queue	Add Chute	Capacity crunch.
Low load, high queue	Reroute	Prevent backlog.
High load, high queue	Add Chute / Reroute / Priority	Emergency intervention.

20. Hyperparameter Grid Search

Grid search tests combinations of alpha and gamma for tabular Q-learning. This answers: which parameter setting learns fastest and achieves the best final reward? In your project, alpha values 0.01, 0.1, 0.5 and gamma values 0.8, 0.95, 0.99 are tested for 2,000 episodes per combination.

This is the manual version of hyperparameter tuning. In future work, Optuna can automate the search more intelligently.

21. Multi-Run Statistical Evaluation

RL results can vary because of randomness. A single good run may be luck. Your project evaluates each policy multiple times with different seeds and reports mean and standard deviation. This is good scientific practice and makes the ranking more trustworthy.

22. Model Persistence

Saving models is important because training is expensive. Your project saves SB3 models as .zip, tabular Q-learning/SARSA as .json, and PyTorch DQN as .pt. This allows reuse, evaluation, and deployment without retraining.

23. Cost-Benefit Analysis

The cost-benefit section translates technical actions into business impact. For example, activating a chute costs money, rerouting has handling cost, priority processing has overtime cost, delays have SLA penalties, and overload incidents have disruption cost.

This makes the reward function more business-realistic. The agent is not just minimizing queue length; it is learning a cost-aware operational policy.

24. Visual Simulation Demo

The visual simulation compares unmanaged operation with RL-managed operation over a 48-hour shift. It shows queue length, chute load, active chutes, actions, and throughput. This is very valuable for explaining the project to non-technical stakeholders.

The simulation demonstrates behaviors such as adding chutes before peak load, rerouting during congestion, removing chutes in low demand, and doing nothing when the system is healthy.

25. Interactive Animation

The notebook includes a matplotlib FuncAnimation with play/pause/scrub controls. This turns the RL process into a live operational dashboard. It also maps visuals back to RL formalism: state, action, transition, reward, policy, and return.

26. Final Explanation of What You Built

You built a complete RL research pipeline for package sorting operations. It starts with a business problem, converts the operation into an MDP, designs a custom environment, trains multiple RL agents, compares them with baselines, explains decisions with heatmaps, evaluates robustness with multiple seeds, saves models, estimates business cost, and visualizes impact.

The strongest technical part is the from-scratch DQN, because it shows deep learning and RL together: the neural network learns Q-values for each action from operational states. The strongest business part is the simulation and cost analysis, because it translates model behavior into operational impact.

27. How to Present This Project

A strong oral explanation can follow this order: 1) problem, 2) why forecasting alone is not enough, 3) RL mapping, 4) environment design, 5) agents tested, 6) DQN deep learning explanation, 7) results, 8) interpretability, 9) business impact, 10) limitations and future work.

One good sentence: "The goal is not only to predict overload, but to learn the best intervention policy that minimizes overload, delay, and operational cost."

28. Limitations and Next Steps

The project is strong as a prototype, but real deployment would need careful validation. The environment must be calibrated with more days of data, operational constraints must be confirmed, and RL recommendations should first run in shadow mode with human override.

Next step	Why
Use more historical data	Four days is useful for prototype, but not enough for production seasonality.
15-minute granularity	More realistic real-time decisions than hourly steps.
LSTM demand prediction as state input	Gives the agent a view of expected future load.
Human-in-the-loop pilot	Safer than direct automation.
Multi-agent RL	One agent per chute/zone for large centers.

29. Glossary

Term	Short explanation
Agent	The learner/decision maker.
Environment	The simulated sorting center.

Term	Short explanation
State	Current situation observed by the agent.
Action	Decision/intervention chosen by the agent.
Reward	Score after an action.
Policy	Strategy for choosing actions.
Q-value	Expected future reward for a state-action pair.
DQN	Deep Q-Network: neural network approximates Q-values.
PPO	Policy optimization method with stable clipped updates.
MaskablePPO	PPO with invalid actions blocked.
Replay buffer	Memory of past experiences used for DQN training.
Target network	Stable copy of Q-network for DQN targets.

End of companion explanation.